

# DecL Cheat Sheet (1/4): agg

An Aggregate compound distribution has the clause structure:

agg <NAME> <LABEL\*> <EXPOSURE> <LAYERS\*> <SEVERITY> <OCC\_RE\*> <FREQUENCY> <AGG\_RE\*> <APPROX\*> <ORIENTATION\*> <TRAILER\*>

Key: <INPUT> user input; lower\_case a DecL keyword; CLAUSE a sub-clause; options a|b|c; inf infinity; \* optional. Build with build('agg ...').

## 1. Name, labels & references

agg <NAME> ...  
sev <NAME> <SEVERITY> *standalone severity*  
... as <LABEL>, ... as "<LABEL>" *display label*

*Names match [a-zA-Z][.\_:~a-zA-Z0-9-]\* and are the identity handle; as adds presentation text. A label may also attach to the exposure, a layer, a severity, a reinsurance layer, and an expense group.*

*Reference: agg.<NAME>, sev.<NAME>, port.<NAME>, dist.<NAME>. Clone / transform: agg <NEW>  
agg.<OLD>, 2 \* agg.X (homog.), 2 @ agg.X (inhomog.), agg.X + 100.*

## 2. Exposure

<EL> loss  
<PREMIUM> premium at <LR> lr  
<EXPOSURE> exposure at <RATE> rate  
<CLAIMS> claims  
dfreq <OUTCOMES> <PROBS\*> *fixed empirical freq*  
<T> years <at <RATE> rate>\* *renewal horizon*

*Outcomes [1 2 3 4] or [2:10:2]; probs [.5 .25 .25] or omitted = equally likely. years pairs only with wait (box 7); its rate is informational and never sizes the count.*

## 3. Limit / layers (optional)

<LIMIT> xs <ATTACHMENT> <LABEL\*>  
tower [<BREAKS>]

*Occurrence limits on ground-up severity, unlimited reinstatements, conditional on attaching by default.*

## 4. Severity (continuous)

sev <DIST> <MEAN> cv <CV>  
sev <DIST> <SHAPE1> <SHAPE2\*>  
sev sev.<NAME> *built-in*  
  
<SCALE> \* SEV + <LOC> *scale & shift*  
ssev <NUM> - SEV *reflect (needs ssev)*  
SEV wts [<W>], SEV wts=<N> *mixture*  
SEV splice [<LB> <UB>] *conditional in layer*  
SEV ! *unconditional (when ATTACH>0)*  
ssev ... *spliced / signed severity*

## 5. Severity (discrete)

dsev <OUTCOMES> <PROBS\*>, dsev ... !  
sev (d|c)histogram xps [<X>] [<P>]  
picks [<X>] [<P>] *match layer picks*  
  
*dsev [1:6] is a fair die; xps gives outcomes and probabilities directly, and attaches only to a histogram.*

## 6. Frequency

poisson, bernoulli, fixed, geometric,  
logarithmic, binomial <P>, negbin <VAR\_MULT>,  
neymana <CLAIMS/OCC>, pascal <CV>  
<CLAIMS/OCC>  
  
mixed <MIX> <SHAPE1> <SHAPE2\*>  
MIX = gamma|delaporte|ig|sig|sichel|beta  
  
FREQ zt zero-truncated; FREQ zm <P0> zero-modified,  
Pr(N=0)=p<sub>0</sub>; trailing ! *pins the realized mean to the exposure clause instead of the base mean*

## 7. Renewal waiting time

wait <SEVERITY> <LABEL\*>  
wait <Y> xs <A> <SEVERITY> <LABEL\*>  
dwait <OUTCOMES> <PROBS\*>, dwait ... !

*The Sparre-Andersen alternative to a frequency clause: the count comes from the waiting-time law over the years horizon. wait takes the full severity mini-language on the time axis. The layered form is conditional by default; ! collapses Pr(W ≤ a) into a zero-wait cluster atom.*

agg R 10 years sev lognorm 50 cv 2 wait  
expon 1

## 8. Reinsurance

occurrence (net of|ceded to) LAYERS  
aggregate (net of|ceded to) LAYERS  
  
LAYER = <LIMIT> xs <ATTACH>  
| <SHARE> so <LIMIT> xs <ATTACH>  
| <PCT> po <LIMIT> xs <ATTACH>  
| <FRAC> of <LIMIT> xs <ATTACH>  
LAYERS = LAYER and LAYER ... *or tower [...]*  
  
*so = share of, po = part of, of = fraction of;  
0 ≤ share ≤ 1. Occurrence applies per claim, aggregate to the annual total. Layers may also carry economics and a label; see page 2.*

## 9. Approximate & orientation

approximate exact (default), approximate sgamma  
shifted gamma, approximate slognorm shifted  
lognormal  
  
*Alias approx. Replaces the freq × sev convolution with one severity matched to the first three aggregate moments.*  
  
*loss (default), payoff. The orientation suffix, last before the trailer: payoff marks an asset-return primitive (more is better) and flips pricing to the dual distortion.*  
  
*tweedie <MEAN> <P> <DISP> a declaration on its own*

## 10. Trailer, vectors, math

**Trailer** note{free text}, tags{cat, property},  
hints{bs=100; log2=17}, doc{{{markdown}}}.  
*Order-free, at most one of each; only doc may carry blank lines and fenced code.*

**Vectorize** [1 2 3] claims; [250 250 500] xs [0 250 500]; slognorm [1 3] cv [.7 1.5] wts [.6 .4]. *Vectors broadcast; layers / exposure zip.*

**Math** 1/2, 3\*\*4, 3^4, exp(2), 12.4%, 10\_000. *Minus binds to the number, so −4<sup>2</sup> = 16; \* is severity scaling, not general arithmetic. Comments # to end of line; statements separate on a blank line or ;.*

aggregate v.1.0.0a171

ease | confidence | precision

© Stephen J. Mildenhall

2026-07-29 11:34



# DecL Cheat Sheet (2/4): reinsurance economics, pnl and xpnl

Page 1 declared the reinsurance *loss* structure. Everything here is *money*: ceded premium, commission, reinstatements, variable rating, and the profit-and-loss statement that books them. A priced layer is LAYER <PREMIUM\*> <CEDE\*> <REINSTATEMENTS\*> <VARIABLE\*> <LABEL\*>, in that order.

## 1. Ceded premium & commission

```
LAYER deposit <AMOUNT>
LAYER rol <RATE> rate on line x share x limit
LAYER rate <FRACTION> of gross premium
```

... cede <FRACTION> ceding commission

The three premium forms are mutually exclusive. *cede* is valid only with a premium; the premium is entered gross of commission.

```
occurrence net of 250 xs 250 deposit 60 cede
20%
```

## 2. Reinstatements

```
reinstatements [0 .5 1 1] explicit multipliers
reinstatements 1 free and 2 at 100%
no reinstatements one annual limit
```

Multipliers  $\alpha_j$  are relative to the base rate on line *r*, so a base premium clause is required. *free* = at 0%. Counts may be digits or the words *one... five*. *m* reinstatements = *m* + 1 annual limits, so *no reinstatements* is one limit — distinct from omitting the clause, which keeps *free* and unlimited.

One occurrence layer, one clause. This is the stochastic-*ceded* property-cat feature.

## 3. Variable rating

```
swing basic <B> lcm <M> <min <LO>* <max
<HI>*
slide <C1> at <LR1> and <C2> at <LR2> ...
pc <SHARE> after <ALLOWANCE>
corridor <SHARE> po <WIDTH> xs <ATTACH>
```

*swing* replaces the deposit / rol / rate premium with the collared affine clip( $b + m \cdot L_{ceded}, lo, hi$ ); *slide* replaces *cede* with a commission sliding on the ceded loss ratio; *pc* is a profit commission on the ceded premium; *corridor* is a loss-ratio band the cedant retains.

At most one variable feature per program (no stacking), and aggregate reinsurance only in this release.

## 4. pnl / xpnl declaration

```
pnl <NAME> <LABEL*> <PREMIUM> less <ENGINE>
<less <EXPENSES>* <TRAILER*>
xpnl <NAME> ... same syntax
```

Profit and loss = premium minus what comes out of the sausage maker. *less* is a dedicated keyword, never -, so the split cannot collide with severity arithmetic (*ssev 20 - lognorm*); a second *less* is the hard anchor introducing expenses.

```
pnl P 1000 premium less agg E 700 loss sev
lognorm 50 cv 2 poisson
```

## 5. The premium head

```
<AMOUNT> premium <LABEL*>
inherit premium <LABEL*>
retro <COLLAR> premium <LABEL*>
```

The premium is the consideration (what you book); the engine's own premium at *lr* is a separate sizing input. *inherit* copies the engine's technical premium (a build error if it has none). *retro* is the account-level retrospective-rating clause, taking the same *basic ... lcm ... min ... max* collar as *swing*.

```
pnl P inherit premium as "gross written"
less agg.Book
```

## 6. The engine source

```
agg <NAME> <AGG-BODY> inline
agg.<NAME> stored aggregate
port.<NAME> stored portfolio (net-net total)
```

A P&L wraps a complete stochastic engine, never an anonymous half-body: the inline form reuses the identical *agg* body from page 1, minus the trailer, which the wrapping *pnl* owns. So every clause on page 1 is available inside a *pnl*.

## 7. Expenses

```
less <TERM> and <TERM> ... <LABEL*> one group
less <GROUP> <GROUP> ... several groups
```

```
TERM = <F> premium expense
      | <F> loss expense
      | <AMOUNT> fixed expense
```

*expense* and *expenses* are both accepted. Terms joined by *and* combine into one reported obligation leg; juxtaposed groups stay separate items. Each group may carry its own as label; the default name is the basis.

```
less 25% premium expense as acquisition 1000
fixed expense as overhead
```

## 8. pnl vs. xpnl

*pnl* the consolidated single-group net view: one *Consideration / Obligation / Margin* card.

*xpnl* the exploded sibling, same syntax: a multi-group walk with one group per step, so you see gross, then each cession, then expenses, as separate blocks that foot to the same margin.

Both return a *PnL*. Reach for *xpnl* to see where the margin went, for *pnl* to see what it is. Both may be portfolio units.

## 9. Worked example

```
xpnl Book 10000 premium
less agg Book_e 10000 premium at 75% lr
10000 xs 0
sev 10.808 * lognorm 1.75
mixed gamma 0.25
aggregate net of 5000 xs 10000
deposit 1500 pc 50% after 10%
less 20% premium expense
note{profit commission on the agg cover}
```

Premium head, engine with an aggregate cession carrying a deposit and a profit commission, then expenses.

# Decl Cheat Sheet (3/4): port and the bivariate family

These declarations build collections and couplings of aggregates. Each component is an ordinary agg (or pn1) declaration, so everything on pages 1 and 2 is available inside them.

## 1. Portfolio (port)

```
port <NAME> <LABEL*> <TRAILER*>
  agg <UNIT1> ...
  agg <UNIT2> ...
  ...
```

A *Portfolio* is a list of independent units sharing a discretization grid; the total is their convolution. Units may be agg or pn1 lines, or references agg. <NAME>. Per-unit reinsurance is set on each unit line.

The portfolio trailer sits before the unit list, so a trailing trailer always binds to the last unit.

## 2. port example

```
port MyBook note{two lines}
  agg A 100 claims sev lognorm 50 cv 2
  poisson
  agg B 200 claims sev gamma 30 cv 1 poisson
```

Access units by name (p.A); *unit\_names\_ex* adds total.

## 3. Bivariate (bv)

```
bivariate <NAME> <SHARED-EXPOSURE>
  agg <A> ...
  agg <B> ...
  copula <KIND> <P*>
  <FREQUENCY*>
```

Abbreviation *bv*. Couples exactly two component severities by a copula and accumulates them with a shared outer frequency via a 2D FFT. The shared exposure may be <N> claims or *dfreq* [...]; the trailing frequency is optional (default poisson). Per-event triggers come from the inner *dfreq* [0 1] [p0 p1] Bernoulli forms.

## 4. Copula clause

```
copula <KIND> <P> one-parameter
copula <KIND> parameter-free
omitted = copula independent
```

```
normal <rho>  $\rho \in (-1, 1)$ 
gumbel <tau>  $\tau \in [0, 1]$  upper tail dependence
clayton <tau>  $\tau \in [0, 1]$  lower tail dependence
fgm <rho_s>  $\rho_s \in [-1/3, 1/3]$ 
independent
```

```
bv M 100 claims agg A ... agg B ... copula
gumbel 0.4
```

## 5. Discrete bivariate (dbvsev)

```
dbvsev [<XS>] [<YS>] [[<ROW>] [<ROW>] ...]
dbvsev [<XS>] [<YS>] uniform on the lattice
dbvsev [[<X> <Y> <P>] ...] sparse triples
```

The 2-D analogue of *dsev*: the joint per-claim probability matrix given directly, *matrix*[i][j] =  $\Pr(X=x_i, Y=y_j)$ . Replaces the two agg bodies and the copula clause. The three surface forms are auto-detected.

```
bv D 10 claims dbvsev [1 2] [3 4] [[.25 .25]
[.25 .25]]
```

## 6. View pairs

```
netceded <AGG>
grossceded <AGG>
grossnet <AGG>
```

One ordinary agg carrying occurrence reinsurance; the keyword names the pair x-then-y out of {gross, ceded, net} (gross leads when present). The two coupled marginals are positions of the same loss, so their dependence is exact, comonotone within the layer. Lets you see the pair jointly, not just their marginals.

```
netceded agg G 100 claims sev lognorm 50 cv
2 occurrence net of 500 xs 500 poisson
```

## 7. Clash

```
clash <NAME> <NA> <NB> <NC> claims
  <LAYERS*> <SEVERITY> component A
  <LAYERS*> <SEVERITY> component B
  <FREQUENCY*>
```

Dependence from interpretable claim counts rather than a copula parameter:  $n_a$  claims hit A,  $n_b$  hit B, and  $n_c$  hit both (the clash). The solver derives the shared event count  $n_0$  and the two per-event Bernoulli triggers  $p_a, p_b$ .

```
clash C 10 8 2 claims 100 xs 0 sev lognorm
50 cv 2 100 xs 0 sev gamma 40 cv 1 poisson
```

# DecL Cheat Sheet (4/4): distortion

A distortion  $g$  defines a coherent risk measure / pricing functional. Declare one by kind and its natural parameters; the kind  $\rightarrow$  parameter mapping lives on each Distortion subclass (`decl_params`), so the grammar holds no per-kind knowledge.

## 1. Declaration

```
distortion <NAME> <KIND> <PARAMS> <TRAILER*>
```

Alias *dist*. <PARAMS> is a flat list of the kind's natural parameters, in the order of the table. Reference a stored distortion as *dist*.<NAME>. A *dist* line carries the same trailer as every other statement, so it can be described, tagged, and documented.

```
distortion D1 ph 0.9
dist D2 wang 0.25 note{market standard}
dist D3 power 0 100 2
```

## 2. Kinds & parameters

One shape parameter:

ph <a> proportional hazard  
wang <lam> Wang / normal shift  
dual <b> dual moment  
tvar <p> tail VaR at  $p$   
ccoc <r> constant cost of capital  
c11 <b> capped log-linear  
clin <slope> capped linear  
lep <r> leverage-equivalent pricing  
ly <r> linear yield

Several parameters:

bitvar <p0> <p1> <w1> weight  $w_1$  on  $p_1$   
beta <a> <b>  $a$  in  $(0, 1]$   
power <x0> <x1> <alpha>

## 3. Combinators

```
distortion <NAME> minimum dist.<A> dist.<B>
...
distortion <NAME> mixture dist.<A> dist.<B>
... wts [<W>]
```

*minimum* = pointwise min of the referenced distortions (min of concaves is concave, so the result is a distortion);  
*mixture* = weighted average, weights via *wts*. Both take distortion references, not numbers.

## 4. Not declarable

*wtdtvar* (weighted TVaR) has no DecL number-list form: it takes parameter vectors of knots and weights. Build it in Python, or reach it through the combinators.

```
Distortion.random_distortion(n_knots=5)
Distortion.mixture([d1, d2], [.6, .4])
```

List the registered kinds with  
`Distortion.available_distortions()`.

## 5. Usage

Apply a distortion to price an aggregate or portfolio:

```
a = build('agg A 100 claims sev lognorm 50
cv 2 poisson')
d = build('distortion D ph 0.9')
a.price(0.99, d)
p.price(0.99, d) total and allocation by unit
```

Calibrate to a target instead of guessing a shape:  
`p.calibrate_distortions(coc=0.1, p=0.99)`.

A distortion satisfies  $g(0) = 0$ ,  $g(1) = 1$ , increasing and concave; the dual is  $g^*(x) = 1 - g(1 - x)$ ; the distorted price is  $\int_0^\infty g(S(x)) dx$ .

## 6. And back again

The reverse question — which distortions are consistent with an observed price — is not a declaration but an analysis. See the *\*Bounds* card: its answers come back as *bitvar* distortions, which are declarable.

```
ab = p.allocation_bounds(p=0.99)
ab.bitvars(1200)
dist D bitvar 0.5 0.99 0.3
```